-- Changes Over Time (Trends) Analysis

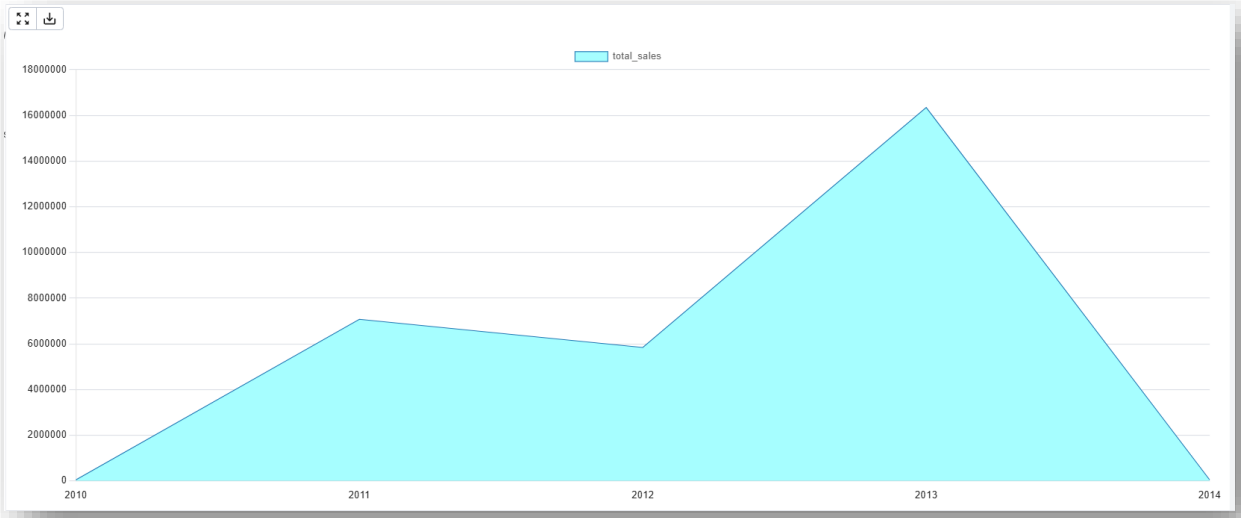
-- Analyze how a measure evolves over time. Helps track trends and identify seasonality in your data.

-- 1. Analyze sales performance over time

-- Yearly analysis

```
select
        extract(year from order_date) as year_is,
        sum(sales_amount) as total_sales,
        count(distinct customer_key) as total_customers,
        sum(quantity) as total_quantity
from gold.fact_sales
where order_date is not null
group by extract(year from order_date)
order by extract(year from order_date) asc;
```

	year_is numeric	total_sales bigint	total_customers bigint	total_quantity bigint
1	2010	43419	14	14
2	2011	7075088	2216	2216
3	2012	5842231	3255	3397
4	2013	16344878	17427	52807
5	2014	45642	834	1970

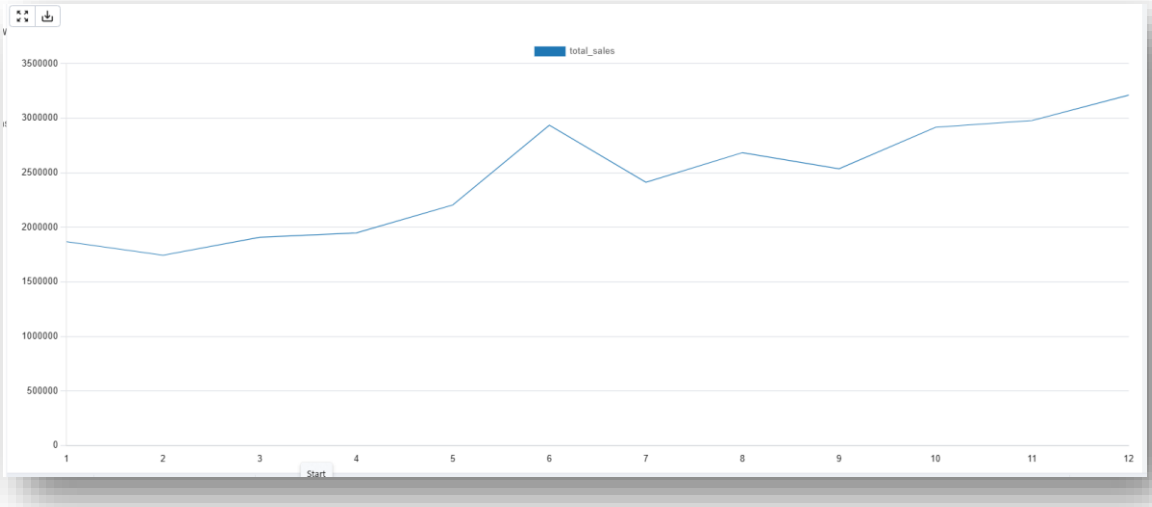


- **2013 was the best year: sales hit 16.3M, more than double 2011 and nearly triple 2012.**
- **2012 had more customers than 2011 but lower sales, so each customer spent less.**
- **2010 and 2014 look like partial years (very few customers), so don't compare them with full years.**

-- Monthly analysis

```
select
        extract(month from order_date) as month_is,
        sum(sales_amount) as total_sales,
        count(distinct customer_key) as total_customers,
        sum(quantity) as total_quantity
from gold.fact_sales
where order_date is not null
group by extract(month from order_date)
order by extract(month from order_date) asc;
```

	month_is numeric	total_sales bigint	total_customers bigint	total_quantity bigint
1	1	1868558	1818	4043
2	2	1744517	1765	3858
3	3	1908375	1982	4449
4	4	1948226	1916	4355
5	5	2204969	2074	4781
6	6	2935883	2430	5573
7	7	2412838	2154	5107
8	8	2684313	2312	5335
9	9	2536520	2210	5070
10	10	2916550	2533	5838
11	11	2979113	2500	5756
12	12	3211396	2656	6239



- **Sales are lowest early in the year (Feb about 1.74M) and highest at the end (Dec about 3.21M), almost double.**
- **Jan to May stays low and slow, between 1.7M and 2.2M.**
- **June jumps sharply to about 2.94M, the best month of the first half.**
- **Oct to Dec is the strongest stretch, going from 2.92M to 3.21M. This is likely the festive or year-end shopping season.**

-- Year-wise Monthly Sales Trend

```
select
        extract(year from order_date) as year_is,
        extract(month from order_date) as month_is,
        sum(sales_amount) as total_sales,
        count(distinct customer_key) as total_customers,
        sum(quantity) as total_quantity
from gold.fact_sales
where order_date is not null
group by extract(year from order_date), extract(month from order_date)
order by extract(year from order_date), extract(month from order_date) asc;
```



	year_is numeric	month_is numeric	total_sales bigint	total_customers bigint	total_quantity bigint
16	2012	3	373478	212	212
17	2012	4	400324	219	219
18	2012	5	358866	207	207
19	2012	6	555142	318	318
20	2012	7	444533	246	246
21	2012	8	523887	294	294
22	2012	9	486149	269	269
23	2012	10	535125	313	313
24	2012	11	537918	324	324
25	2012	12	624454	354	483
26	2013	1	857758	627	1677
27	2013	2	771218	1373	3454
28	2013	3	1049732	1631	4087
29	2013	4	1045860	1564	3979
30	2013	5	1284456	1719	4400
31	2013	6	1642948	1948	5025
32	2013	7	1371595	1796	4673
33	2013	8	1545910	1898	4848
34	2013	9	1447324	1832	4616
35	2013	10	1673261	2073	5304
36	2013	11	1780688	2036	5224
37	2013	12	1874128	2133	5520
38	2014	1	45642	834	1970

-- Cumulative Analysis

-- Aggregate the data porgressively over time. Helps to understand whether our business is growing or declining.

-- 1. Calculate the total sales per month and the running total of sales over time.

```
select
*,
sum(total_sales) over (order by year_is, month_is) as running_total_sales
from (  -- Subquery
                select
                        extract(year from order_date) as year_is,
                        extract(month from order_date) as month_is,
                        sum(sales_amount) as total_sales
                from gold.fact_sales
                where order_date is not null
                group by extract(year from order_date), extract(month from order_date)
  ) as Sub;
```

	year_is numeric	month_is numeric	total_sales bigint	running_total_sales numeric
1	2010	12	43419	43419
2	2011	1	469795	513214
3	2011	2	466307	979521
4	2011	3	485165	1464686
5	2011	4	502042	1966728
6	2011	5	561647	2528375
7	2011	6	737793	3266168
8	2011	7	596710	3862878
9	2011	8	614516	4477394
10	2011	9	603047	5080441
11	2011	10	708164	5788605
12	2011	11	660507	6449112
13	2011	12	669395	7118507
14	2012	1	495363	7613870
15	2012	2	506992	8120862
16	2012	3	373478	8494340
17	2012	4	400324	8894664
18	2012	5	358866	9253530
19	2012	6	555142	9808672
20	2012	7	444533	10253205
21	2012	8	523887	10777092
22	2012	9	486149	11263241
23	2012	10	535125	11798366

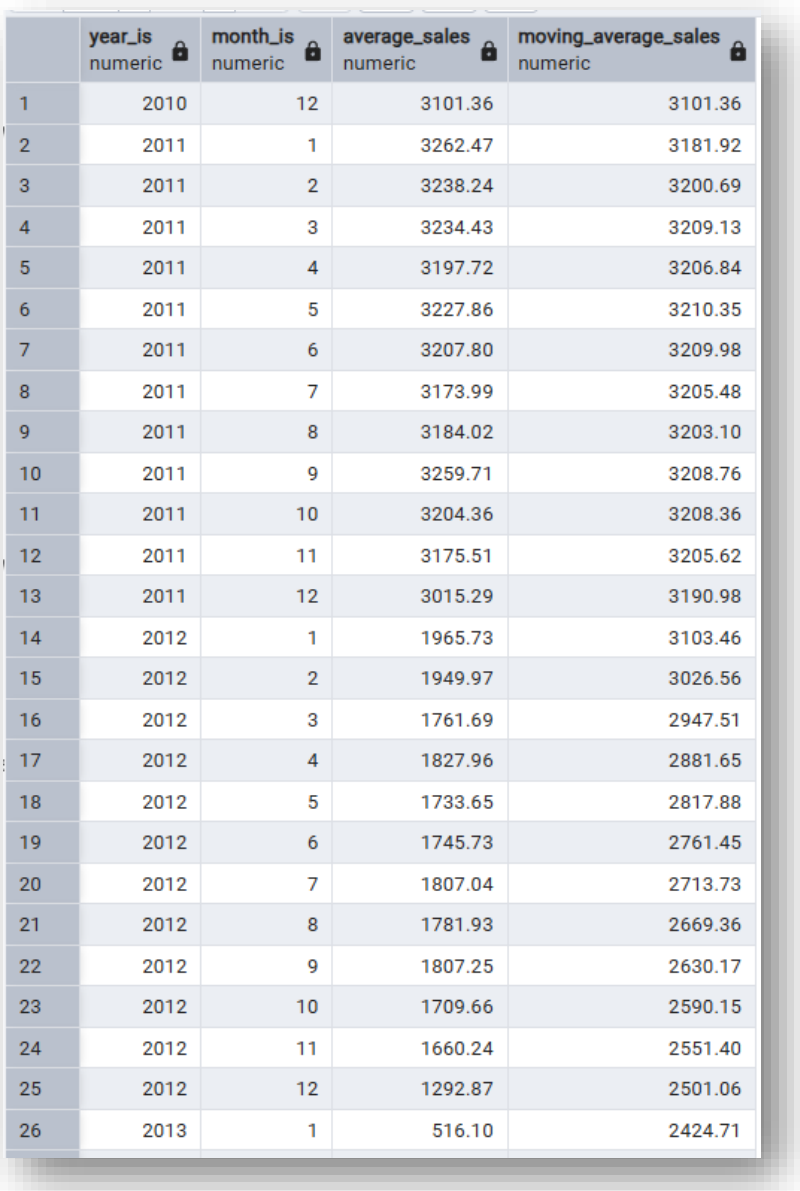
-- 2. Calculate the average sales per month and the moving average of sales over time.

```
select

*,

round(avg(average_sales) over(order by year_is, month_is), 2) as moving_average_sales
```

```
from (  -- Subquery
                select
                        extract(year from order_date) as year_is,
                        extract(month from order_date) as month_is,
                        round(avg(sales_amount), 2) as average_sales
                from gold.fact_sales
                where order_date is not null
                group by extract(year from order_date), extract(month from order_date)
    ) as Sub;
```



	year_is numeric	month_is numeric	average_sales numeric	moving_average_sales numeric
1	2010	12	3101.36	3101.36
2	2011	1	3262.47	3181.92
3	2011	2	3238.24	3200.69
4	2011	3	3234.43	3209.13
5	2011	4	3197.72	3206.84
6	2011	5	3227.86	3210.35
7	2011	6	3207.80	3209.98
8	2011	7	3173.99	3205.48
9	2011	8	3184.02	3203.10
10	2011	9	3259.71	3208.76
11	2011	10	3204.36	3208.36
12	2011	11	3175.51	3205.62
13	2011	12	3015.29	3190.98
14	2012	1	1965.73	3103.46
15	2012	2	1949.97	3026.56
16	2012	3	1761.69	2947.51
17	2012	4	1827.96	2881.65
18	2012	5	1733.65	2817.88
19	2012	6	1745.73	2761.45
20	2012	7	1807.04	2713.73
21	2012	8	1781.93	2669.36
22	2012	9	1807.25	2630.17
23	2012	10	1709.66	2590.15
24	2012	11	1660.24	2551.40
25	2012	12	1292.87	2501.06
26	2013	1	516.10	2424.71

-- Performance Analysis

-- Comparing the current value to a target value. Helps measure success and compare performance.

-- 1. Analyze the yearly performance of products by comparing each product's sales to both its average sales performance and the previous year's sales.

```
with yearly_product_sales as
(
select
        extract(year from s.order_date) as year_is,
        p.product_name,
        sum(s.sales_amount) as current_sales
from gold.fact_sales as s
left join gold.dim_products as p
        on s.product_key = p.product_key
where extract(year from s.order_date) is not null
group by extract(year from s.order_date), p.product_name
)

select
        year_is,
        product_name,
        current_sales,
        round(avg(current_sales) over(partition by product_name), 2) as avg_sales,
        current_sales - round(avg(current_sales) over(partition by product_name), 2) as diff_avg,
                case when current_sales - round(avg(current_sales) over(partition by product_name), 2) > 0 then 'Above Avg'
```

```
        when current_sales - round(avg(current_sales) over(partition by
product_name), 2) < 0 then 'Below Avg'
            else 'Same Avg'
            end as avg_flag,
    lag(current_sales) over(partition by product_name order by year_is asc) as
previous_year_sales,
    current_sales - lag(current_sales) over(partition by product_name order by
year_is asc) as diff_prev_year_sales,
        case when current_sales - lag(current_sales) over(partition by
product_name order by year_is asc) > 0 then 'Increase'
        when current_sales - lag(current_sales) over(partition by
product_name order by year_is asc) < 0 then 'Decrease'
            else 'No Change'
            end as py_diff_flag
from yearly_product_sales
order by product_name, year_is
```

	year_is numeric	product_name character varying (50)	current_sales bigint	avg_sales numeric	diff_avg numeric	avg_flag text	previous_year_sales bigint	diff_prev_year_sales bigint	py_diff_flag text
1	2012	All-Purpose Bike Stand	159	13197.00	-13038.00	Below Avg	[null]	[null]	No Change
2	2013	All-Purpose Bike Stand	37683	13197.00	24486.00	Above Avg	159	37524	Increase
3	2014	All-Purpose Bike Stand	1749	13197.00	-11448.00	Below Avg	37683	-35934	Decrease
4	2012	AWC Logo Cap	72	6570.00	-6498.00	Below Avg	[null]	[null]	No Change
5	2013	AWC Logo Cap	18891	6570.00	12321.00	Above Avg	72	18819	Increase
6	2014	AWC Logo Cap	747	6570.00	-5823.00	Below Avg	18891	-18144	Decrease
7	2013	Bike Wash - Dissolver	6960	3636.00	3324.00	Above Avg	[null]	[null]	No Change
8	2014	Bike Wash - Dissolver	312	3636.00	-3324.00	Below Avg	6960	-6648	Decrease
9	2013	Classic Vest- L	11968	6240.00	5728.00	Above Avg	[null]	[null]	No Change
10	2014	Classic Vest- L	512	6240.00	-5728.00	Below Avg	11968	-11456	Decrease
11	2013	Classic Vest- M	11840	6368.00	5472.00	Above Avg	[null]	[null]	No Change
12	2014	Classic Vest- M	896	6368.00	-5472.00	Below Avg	11840	-10944	Decrease
13	2012	Classic Vest- S	64	3648.00	-3584.00	Below Avg	[null]	[null]	No Change
14	2013	Classic Vest- S	10368	3648.00	6720.00	Above Avg	64	10304	Increase
15	2014	Classic Vest- S	512	3648.00	-3136.00	Below Avg	10368	-9856	Decrease
16	2012	Fender Set - Mountain	110	15554.00	-15444.00	Below Avg	[null]	[null]	No Change
17	2013	Fender Set - Mountain	44484	15554.00	28930.00	Above Avg	110	44374	Increase
18	2014	Fender Set - Mountain	2068	15554.00	-13486.00	Below Avg	44484	-42416	Decrease
19	2012	Half-Finger Gloves- L	24	3544.00	-3520.00	Below Avg	[null]	[null]	No Change
20	2013	Half-Finger Gloves- L	10248	3544.00	6704.00	Above Avg	24	10224	Increase

-- Part-to-Whole (Proportional)

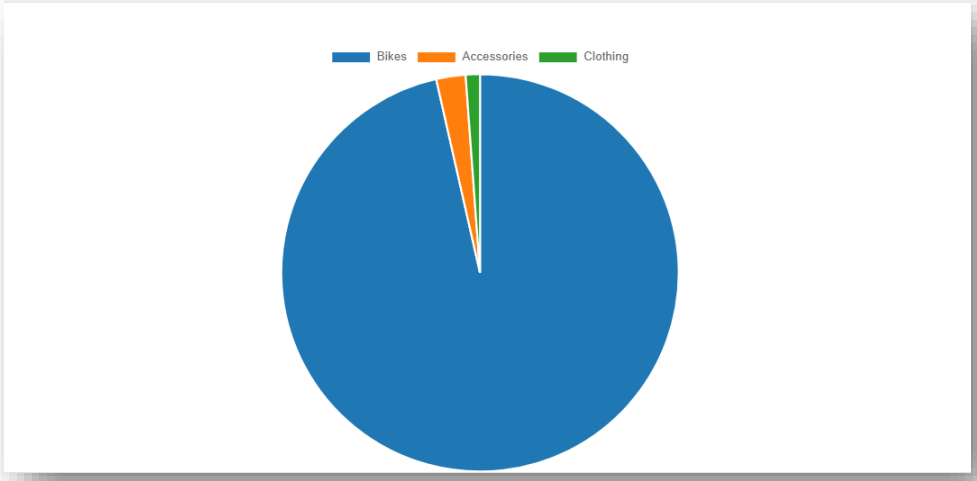
-- Analyze how an individual part is performing compared to the overall, allowing us to understand which category has the greatest impact on the business.

-- 1. Which category contribute the most to overall sales

```
with sales_by_category as
(
select
        p.category,
        sum(s.sales_amount) as total_sales
from gold.fact_sales as s
left join gold.dim_products as p
        on s.product_key = p.product_key
group by p.category
)

select
        category,
        total_sales,
        sum(total_sales) over() as overall_sales,
        concat(round((total_sales / sum(total_sales) over()) * 100, 2), ' %') as
percentage_share
from sales_by_category
```

	category character varying (50)	total_sales bigint	overall_sales numeric	percentage_share text
1	Bikes	28316272	29356250	96.46 %
2	Accessories	700262	29356250	2.39 %
3	Clothing	339716	29356250	1.16 %



1. Bikes dominate total sales, contributing 96.46% (28.32 million) of overall sales, making them the primary revenue driver.

2. Accessories contribute 2.39% and Clothing just 1.16%, indicating significant sales concentration in the Bikes category.

-- Data Segmentation

-- Group the data based on a specific range. Helps understand the correlation between two measures.

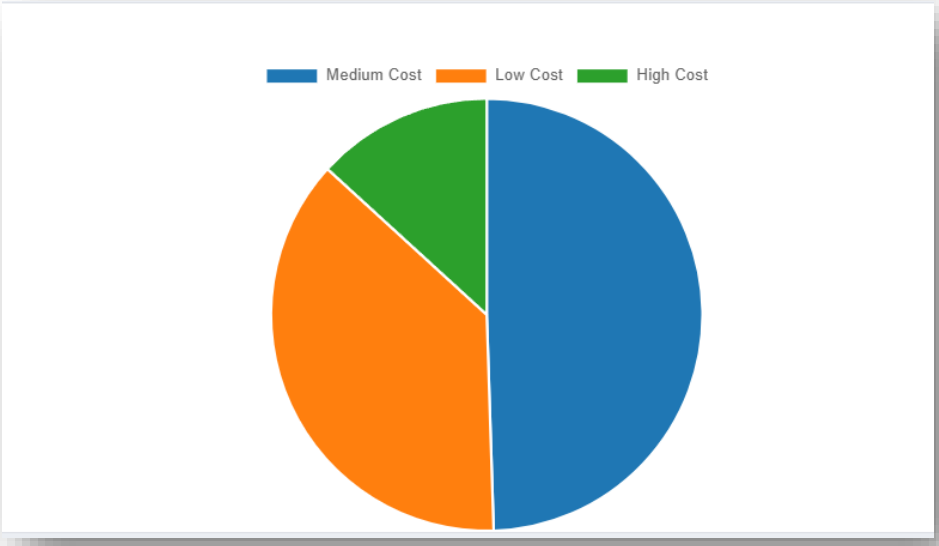
-- 1. Segment products into cost ranges and count how many products fall into each segment.

```
with cost_segmentation as
(
select
        product_key,
        product_name,
        cost,
        case when cost < 100 then 'Low Cost'
            when cost < 1000 then 'Medium Cost'
                else 'High Cost'
```

```
        end as segmentation
from gold.dim_products
)

select
        segmentation,
        count(product_key) as total_products
from cost_segmentation
group by segmentation
order by total_products desc;
```

	segmentation text	total_products bigint
1	Medium Cost	146
2	Low Cost	110
3	High Cost	39



1. **Medium Cost products dominate the inventory with 146 products, making it the largest product segment.**
2. **Low Cost includes 110 products, while High Cost has only 39, indicating a smaller proportion of high-priced products.**

-- 2. Group customer into three segments based on their spending behavior:

-- VIP: At least 12 months of history and spending more than 5000

-- Regular: At least 12 months of history but spending 5000 or less

-- New: Lifespan less than 12 months

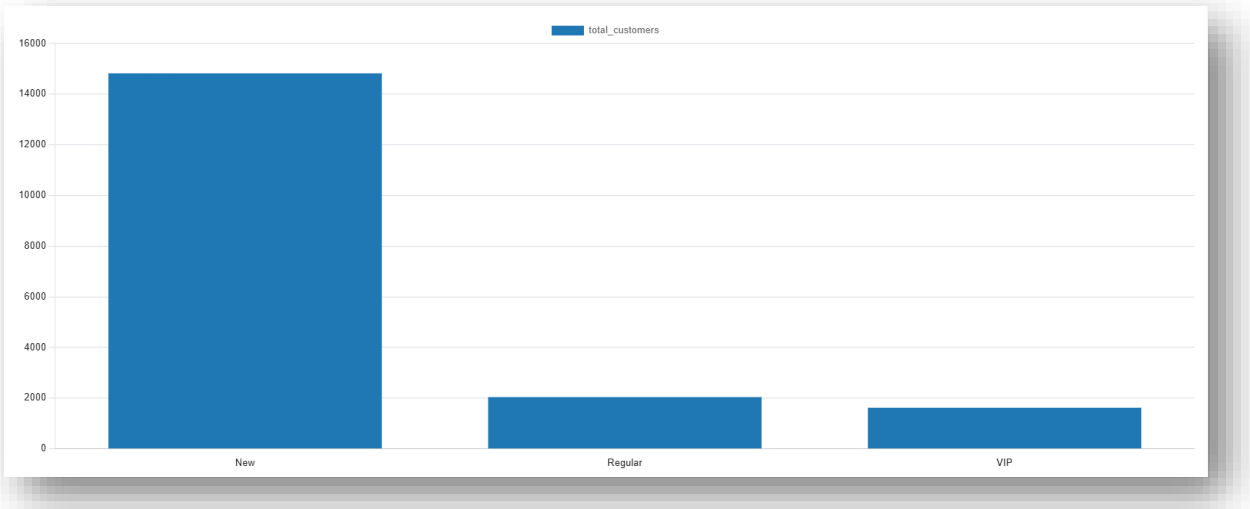
-- And find the total numbers of customers by each group.

```
with customer_segmentation as
(
select
        c.customer_key,
        sum(s.sales_amount) as total_spending,
        min(s.order_date) as first_order_date,
        max(s.order_date) as last_order_date,
        age(max(s.order_date), min(s.order_date)) as time_span,
        case when age(max(s.order_date), min(s.order_date)) >= interval '1 years' and
sum(s.sales_amount) > 5000 then 'VIP'
            when age(max(s.order_date), min(s.order_date)) >= interval '1 years' and
sum(s.sales_amount) <= 5000 then 'Regular'
                else 'New'
                end as customer_segment
from gold.fact_sales as s
left join gold.dim_customers as c
on s.customer_key = c.customer_key
group by c.customer_key
```

```
)


select
        customer_segment,
        count(customer_key) as total_customers
from customer_segmentation
group by customer_segment
order by total_customers desc;
```

	customer_segment text	total_customers bigint
1	New	14823
2	Regular	2040
3	VIP	1621



- **Most customers are New: 14,823 of 18,484 (about 80%) have less than 12 months of history, so the customer base is mostly recent buyers.**
- **Only 1,621 customers (about 9%) are VIP, so focus retention on them and on moving New customers to Regular.**